

HDOC DAY-33

Distributive Properties of Matrix Multiplication

Aim

To write a **menu-driven C program** using **separate functions** and **Variable Length Arrays (VLA)** to verify:

1. **Left Distributive Property**

$$A(B+C)=(AB)+(AC)$$

2. **Right Distributive Property**

$$(A+B)C=(AC)+(BC)$$

3. Exit

1. Algorithm

Main Algorithm

1. Start.
2. Display the menu:
 - Verify Left Distributive Property
 - Verify Right Distributive Property
 - Exit
3. Read the user's choice.
4. Use a `switch` statement:
 - If choice is **1**, call `leftDistributive()`.
 - If choice is **2**, call `rightDistributive()`.
 - If choice is **3**, terminate the program.
 - Otherwise, display "Invalid choice!".
5. Repeat the menu until the user selects Exit.
6. Stop.

Algorithm for Left Distributive Property

Given:

- $A \rightarrow m \times n$
- $B \rightarrow n \times p$
- $C \rightarrow n \times p$

Verify:

$$A(B+C)=(AB)+(AC)$$

1. Read m , n , and p .
2. Declare VLA matrices A , B , and C .
3. Read all elements of A , B , and C .
4. Calculate: $S=B+C$
5. Calculate the left-hand side: $LHS=A * S=A(B+C)$
6. Calculate: $AB=A*B$
7. Calculate: $AC=A*C$
8. Calculate the right-hand side: $RHS=AB+AC$
9. Display LHS and RHS matrices.
10. Compare corresponding elements.
11. If all elements are equal, display that the **Left Distributive Property is verified**; otherwise, display that it is not verified.

Algorithm for Right Distributive Property

Given:

- $A \rightarrow m \times n$
- $B \rightarrow m \times n$
- $C \rightarrow n \times p$

Verify:

$$(A+B)C=(AC)+(BC)$$

1. Read m , n , and p .
 2. Declare VLA matrices A , B , and C .
 3. Read their elements.
 4. Calculate: $S=A+B$
 5. Calculate: $LHS=S*C=(A+B)C$
 6. Calculate: $AC=A*C$
 7. Calculate: $BC=B*C$
 8. Calculate: $RHS=AC+BC$
 9. Display LHS and RHS.
 10. Compare corresponding elements.
 11. If both matrices are equal, display that the **Right Distributive Property is verified**.
-

2. C Program

```
#include <stdio.h>

// Function to read a matrix
void readMatrix(int r, int c, int a[r][c])
{
    for (int i = 0; i < r; i++)
    {
        for (int j = 0; j < c; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }
}

// Function to display a matrix
void displayMatrix(int r, int c, int a[r][c])
{
    for (int i = 0; i < r; i++)
    {
        for (int j = 0; j < c; j++)
        {
            printf("%d\t", a[i][j]);
        }
        printf("\n");
    }
}

// Function for matrix addition
void addMatrix(int r, int c,
               int a[r][c], int b[r][c], int sum[r][c])
{
    for (int i = 0; i < r; i++)
    {
        for (int j = 0; j < c; j++)
        {
            sum[i][j] = a[i][j] + b[i][j];
        }
    }
}

// Function for matrix multiplication
void multiplyMatrix(int r1, int c1, int c2,
                   int a[r1][c1],
                   int b[c1][c2],
                   int result[r1][c2])
{
    for (int i = 0; i < r1; i++)
    {
        for (int j = 0; j < c2; j++)
        {
            result[i][j] = 0;

            for (int k = 0; k < c1; k++)
```

```

        {
            result[i][j] += a[i][k] * b[k][j];
        }
    }
}

// Function to compare two matrices
int compareMatrix(int r, int c,
                  int a[r][c], int b[r][c])
{
    for (int i = 0; i < r; i++)
    {
        for (int j = 0; j < c; j++)
        {
            if (a[i][j] != b[i][j])
                return 0;
        }
    }

    return 1;
}

// Function to verify left distributive property
void leftDistributive()
{
    int m, n, p;

    printf("\nEnter m, n and p: ");
    scanf("%d %d %d", &m, &n, &p);

    int A[m][n];
    int B[n][p];
    int C[n][p];

    int sumBC[n][p];
    int LHS[m][p];

    int AB[m][p];
    int AC[m][p];
    int RHS[m][p];

    printf("Enter elements of matrix A (%d x %d):\n", m, n);
    readMatrix(m, n, A);

    printf("Enter elements of matrix B (%d x %d):\n", n, p);
    readMatrix(n, p, B);

    printf("Enter elements of matrix C (%d x %d):\n", n, p);
    readMatrix(n, p, C);

    // B + C
    addMatrix(n, p, B, C, sumBC);

    // A * (B + C)
    multiplyMatrix(m, n, p, A, sumBC, LHS);
}

```

```

// A * B
multiplyMatrix(m, n, p, A, B, AB);

// A * C
multiplyMatrix(m, n, p, A, C, AC);

// (A * B) + (A * C)
addMatrix(m, p, AB, AC, RHS);

printf("\nA * (B + C):\n");
displayMatrix(m, p, LHS);

printf("\n(A * B) + (A * C):\n");
displayMatrix(m, p, RHS);

if (compareMatrix(m, p, LHS, RHS))
    printf("\nLeft Distributive Property is verified.\n");
else
    printf("\nLeft Distributive Property is not verified.\n");
}

// Function to verify right distributive property
void rightDistributive()
{
    int m, n, p;

    printf("\nEnter m, n and p: ");
    scanf("%d %d %d", &m, &n, &p);

    int A[m][n];
    int B[m][n];
    int C[n][p];

    int sumAB[m][n];
    int LHS[m][p];

    int AC[m][p];
    int BC[m][p];
    int RHS[m][p];

    printf("Enter elements of matrix A (%d x %d):\n", m, n);
    readMatrix(m, n, A);

    printf("Enter elements of matrix B (%d x %d):\n", m, n);
    readMatrix(m, n, B);

    printf("Enter elements of matrix C (%d x %d):\n", n, p);
    readMatrix(n, p, C);

    // A + B
    addMatrix(m, n, A, B, sumAB);

    // (A + B) * C
    multiplyMatrix(m, n, p, sumAB, C, LHS);

    // A * C
    multiplyMatrix(m, n, p, A, C, AC);

```

```

// B * C
multiplyMatrix(m, n, p, B, C, BC);

// (A * C) + (B * C)
addMatrix(m, p, AC, BC, RHS);

printf("\n(A + B) * C:\n");
displayMatrix(m, p, LHS);

printf("\n(A * C) + (B * C):\n");
displayMatrix(m, p, RHS);

if (compareMatrix(m, p, LHS, RHS))
    printf("\nRight Distributive Property is verified.\n");
else
    printf("\nRight Distributive Property is not verified.\n");
}

// Main function
int main()
{
    int choice;

    do
    {
        printf("\n===== MENU =====\n");
        printf("1. Verify Left Distributive Property\n");
        printf("2. Verify Right Distributive Property\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                leftDistributive();
                break;

            case 2:
                rightDistributive();
                break;

            case 3:
                printf("Exiting program...\n");
                break;

            default:
                printf("Invalid choice! Please enter 1, 2 or 3.\n");
        }
    } while (choice != 3);

    return 0;
}

```

3. Test Case Table

Test Case	Menu Choice	Dimensions	Input Type	Expected Result
TC-1	1	m=2, n=2, p=2	Normal positive matrices	Left distributive property verified
TC-2	2	m=2, n=2, p=2	Normal positive matrices	Right distributive property verified
TC-3	1	m=2, n=3, p=2	Rectangular matrices	Left distributive property verified
TC-4	2	m=2, n=3, p=2	Rectangular matrices	Right distributive property verified
TC-5	1	m=2, n=2, p=2	Zero matrix included	Left distributive property verified
TC-6	2	m=2, n=2, p=2	Negative elements included	Right distributive property verified
TC-7	4	—	Invalid menu choice	Invalid choice message
TC-8	3	—	Exit	Program terminates

4. Test Case Simulation and Evaluation

Test Case 1 – Left Distributive Property

```

(kali㉿kali)-[~/Desktop]
$ cc day33.c

(kali㉿kali)-[~/Desktop]
$ ./a.out

===== MENU =====
1. Verify Left Distributive Property
2. Verify Right Distributive Property
3. Exit
Enter your choice: 1

Enter m, n and p: 2 2 2
Enter elements of matrix A (2 x 2):
1 2
3 4
Enter elements of matrix B (2 x 2):
4 5
6 7
Enter elements of matrix C (2 x 2):
8 9
0 1

A * (B + C):
24      30
60      74

(A * B) + (A * C):
24      30
60      74

Left Distributive Property is verified.

```

Evaluation

Both sides produce the same matrix:

$$A(B+C)=(AB)+(AC)$$

Therefore, the program correctly verifies the **left distributive property**.

Test Case 2 – Right Distributive Property

```

===== MENU =====
1. Verify Left Distributive Property
2. Verify Right Distributive Property
3. Exit
Enter your choice: 2

Enter m, n and p: 2 2 2
Enter elements of matrix A (2 x 2):
3 7
0 9
Enter elements of matrix B (2 x 2):
8 1
5 3
Enter elements of matrix C (2 x 2):
6 1
0 8

(A + B) * C:
66      75
30      101

(A * C) + (B * C):
66      75
30      101

Right Distributive Property is verified.

```

Evaluation

The matrices obtained on both sides are identical:

$$(A+B)C=(AC)+(BC)$$

Hence, the program correctly verifies the **right distributive property**.

Test Case 3 – Rectangular Matrices for Left Distributive Property

Use:

- $m = 2$
- $n = 3$
- $p = 2$

This gives:

- $A = 2 \times 3$
- $B = 3 \times 2$
- $C = 3 \times 2$

```

===== MENU =====
1. Verify Left Distributive Property
2. Verify Right Distributive Property
3. Exit
Enter your choice: 1

Enter m, n and p: 2 3 2
Enter elements of matrix A (2 x 3):
4 7 8
9 2 0
Enter elements of matrix B (3 x 2):
2 3 6
1 1 0
Enter elements of matrix C (3 x 2):
6 9 9
3 3 8

A * (B + C):
169    140
102    116

(A * B) + (A * C):
169    140
102    116

Left Distributive Property is verified.

```

The program successfully performs $B+C$, followed by matrix multiplication. Both $A(B+C)$ and $(AB) + (AC)$ produce matrices of dimension 2×2 , and their corresponding elements are equal.

Result: Left distributive property is verified.

Test Case 4 – Rectangular Matrices for Right Distributive Property

Use:

- $A = 2 \times 3$
- $B = 2 \times 3$
- $C = 3 \times 2$

```

===== MENU =====
1. Verify Left Distributive Property
2. Verify Right Distributive Property
3. Exit
Enter your choice: 2

Enter m, n and p: 2 3 2
Enter elements of matrix A (2 x 3):
9 9 0
0 9 9
Enter elements of matrix B (2 x 3):
7 7 1
1 4 5
Enter elements of matrix C (3 x 2):
6 7
6 7
6 7

(A + B) * C:
198    231
168    196

(A * C) + (B * C):
198    231
168    196

Right Distributive Property is verified.

```

The resulting LHS and RHS matrices are equal.

Result: Right distributive property is verified.

Test Case 5 – Zero Matrix

```

===== MENU =====
1. Verify Left Distributive Property
2. Verify Right Distributive Property
3. Exit
Enter your choice: 1

Enter m, n and p: 2 2 2
Enter elements of matrix A (2 x 2):
6 6
3 4
Enter elements of matrix B (2 x 2):
0 0
0 0
Enter elements of matrix C (2 x 2):
8 9
2 1

A * (B + C):
60    60
32    31

(A * B) + (A * C):
60    60
32    31

Left Distributive Property is verified.

```

Result: The program correctly handles zero elements and verifies the left distributive property.

Test Case 6 – Negative Elements

```
===== MENU =====
1. Verify Left Distributive Property
2. Verify Right Distributive Property
3. Exit
Enter your choice: 2

Enter m, n and p: 2 2 2
Enter elements of matrix A (2 x 2):
-6 -7
0 -1
Enter elements of matrix B (2 x 2):
5 6
0 0
Enter elements of matrix C (2 x 2):
-3 2
8
-6

(A + B) * C:
-5 4
-8 6

(A * C) + (B * C):
-5 4
-8 6

Right Distributive Property is verified.
```

Matrices containing negative integers are entered for A, B, and C.

The program performs addition and multiplication using integer arithmetic. Negative values do not affect the distributive law, and the LHS and RHS matrices remain equal.

Result: Right distributive property is successfully verified even with negative matrix elements.

Test Case 7 – Invalid Choice

```
===== MENU =====
1. Verify Left Distributive Property
2. Verify Right Distributive Property
3. Exit
Enter your choice: 8
Invalid choice! Please enter 1, 2 or 3.
```

Evaluation

The default case of the switch statement handles an invalid menu choice correctly. The program then displays the menu again instead of terminating.

Test Case 8 – Exit

```
===== MENU =====
1. Verify Left Distributive Property
2. Verify Right Distributive Property
3. Exit
Enter your choice: 3
Exiting program ...
```

Evaluation

When option 3 is selected, the loop condition becomes false and the program terminates successfully.

Conclusion

The program successfully uses **Variable Length 2D Arrays, separate functions, matrix addition, matrix multiplication, matrix comparison, and switch-case** to verify:

$$\{A(B+C)=AB+AC\}$$

$$\{(A+B)C=AC+BC\}$$

Hence, both the **left and right distributive properties of matrix multiplication over matrix addition** are verified.